ST2: Week 4 Tutorial & Computer Lab

Task 1

```
ENEMY TURN
Enemy selected:  special
Player health: 35
Enemy health: 30

PLAYER TURN
Select move...
normal
You selected:  normal
Player health: 35
Enemy health: 10

ENEMY TURN
Enemy selected:  last stand
Player health: 5
Enemy health: -5

GAME OVER: Player wins.
```

```
PLAYER TURN
Select move...
normal
You selected:  normal
Player health: 10
Enemy health: 980

ENEMY TURN
Enemy selected:  normal
Player health: -10
Enemy health: 980

GAME OVER: Enemy wins.
```

```
└─ python rpg.py
PLAYER TURN
Select move...

Invalid move. Try again.
Select move...
normal
You selected:  normal
Player health: 0
Enemy health: -20

DRAW.
```

Task 2

```
python algorithms_project_file.py
99999
-1
1
8
Linear search execution time: 3.037987 seconds
Binary search execution time: 0.000424 seconds
Sorted array: [11, 12, 22, 25, 64]
Sorted array: [1, 11, 12, 22, 64]
Bubble Sort Execution Time: 6.731025 seconds
Insertion Sort Execution Time: 1.882036 seconds
Binary Search (With Sorting) Execution Time: 2.114844 seconds
Linear Search Execution Time: 0.000235 seconds
```

Linear Search

Observations:

- Target 99999 in a list of 1,000,000 elements → found at index 99999.
- Target -999 → not found, returned -1.
- Execution time for a large list: ~3 seconds.

Reasoning:

- Linear search checks every element sequentially until the target is found or the end of the list is reached.
- Its time complexity is O(n), meaning that as the size of the dataset increases, the execution time increases linearly.
- In the Worst-case scenario occurs when the target is at the end or does not exist in the list.

Practical Implications:

- Linear search is simple to implement and works on any list, sorted or unsorted.
- Efficient for small datasets or one-off searches, but becomes very inefficient for large datasets.
- Because it does not rely on the order of elements, it is robust but not scalable.

Binary search clearly outperforms linear search for sorted lists because it eliminates half the remaining list with each comparison. Linear search is simple but inefficient for large datasets.

Binary Search

Observations:

- Target 1 → found at index 1.
- Target 555 → found at index 8.
- Execution time: 0.000424 seconds (for a list of 1,000,000 elements).

Reasoning:

- Binary search requires a sorted list.
- It repeatedly divides the search interval in half, comparing the target with the middle element.
- Its time complexity is O(log n), which grows very slowly compared to the list size.
- For a list of 1,000,000 elements, it only needs about 20 comparisons (log2(1,000,000) ≈ 20) to locate the target.

Practical Implications:

- Binary search is extremely efficient for large, sorted datasets.
- Ideal for applications where many searches will be performed on the same sorted data (e.g., database lookups, searching sorted logs).
- Not suitable for unsorted data, as it can return incorrect results if sorting is not applied first.

Binary Search on Unsorted Data (With Sorting)

Observations

- Binary search combined with insertion sort: ~2.1 seconds.
- Linear search on unsorted list: ~0.0002 seconds.

Reasoning

- Sorting the list before performing binary search adds significant overhead.
- Even though binary search itself is fast, the sorting step dominates the runtime for a single search.
- The total time becomes sorting time + search time, which can make linear search faster in this scenario.

Practical Implications

- Sorting before search is only beneficial if the list is reused for multiple searches.
- For one-off searches on unsorted data, linear search is generally more efficient despite its O(n) complexity.
- This illustrates an important principle in algorithm selection: theoretical efficiency must be considered in a practical context, including the cost of pre-processing.

Key takeaways

1. Binary search is far more efficient than linear search on large sorted lists, thanks to its logarithmic scaling.
2. Linear search is better for unsorted lists if you only need to search once or sorting is expensive.
3. Sorting overhead matters: even the “fastest” search algorithm may perform worse if preparation steps (like sorting) are required.
4. Algorithm selection is context-dependent: the size of data, number of searches, and whether data is already sorted are all important factors.

Task 3

```
python algorithms_project_file.py
99999
-1
1
8
Linear search execution time: 3.037987 seconds
Binary search execution time: 0.000424 seconds
Sorted array: [11, 12, 22, 25, 64]
Sorted array: [1, 11, 12, 22, 64]
Bubble Sort Execution Time: 6.731025 seconds
Insertion Sort Execution Time: 1.882036 seconds
Binary Search (With Sorting) Execution Time: 2.114844 seconds
Linear Search Execution Time: 0.000235 seconds
```

Insertion Sort

Observations

- Example list [64, 1, 12, 22, 11] → sorted to [1, 11, 12, 22, 64].
- Execution time for 10,000 random integers: ~1.88 seconds.

Reasoning

- Insertion sort builds the sorted list incrementally by taking one element at a time and inserting it into its correct position in the sorted portion.
- Time complexity:
  - Best case (already sorted): $O(n)$ → minimal comparisons.
  - Worst case (reverse order): $O(n^2)$ → maximum comparisons and shifts.
- Efficient for small or partially sorted datasets, as it avoids unnecessary comparisons for already sorted elements.

Practical Implications

- Good for small lists, lists that are nearly sorted, or when inserting new items into an already sorted array.
- Becomes slow for very large, random datasets, but still faster than bubble sort in most cases.

Bubble Sort

Observations

- Example list [64, 25, 12, 22, 11] → sorted to [11, 12, 22, 25, 64].
- Execution time for 10,000 random integers: ~6.73 seconds.

Reasoning

- Bubble sort repeatedly compares adjacent elements and swaps them if they are out of order.
- Uses a swapped flag to detect when no swaps are needed, which slightly optimizes performance.
- Time complexity:
    - Best case (already sorted): $O(n)$ → early exit with swapped flag.
    - Worst case: $O(n^2)$ → still slow for large datasets due to repeated passes.

Practical Implications

- Mainly useful as a teaching tool to understand sorting mechanics.
- Not recommended for large datasets, even with the optimization flag.

Bubble Sort vs Insertion Sort Comparison

:

Algorithm	Time (10,000 elements)	Notes
Bubble Sort	~6.73 s	Slower due to many unnecessary swaps.
Insertion Sort	~1.88 s	Faster because it minimizes moves for nearly sorted elements.

Comment

- Insertion sort generally outperforms bubble sort for random data.
- Bubble sort is more repetitive and performs poorly on larger arrays.

Task 4

Linear Search + Sorting

- Linear search itself does not require sorted data, so combining it with sorting is redundant.
- Execution times increase significantly because the sorting step dominates.
- Combining sorting + linear search adds unnecessary overhead.
- Linear search is already O(n), and sorting adds $O(n^2)$ (bubble/insertion), making it far slower.

Binary Search + Sorting

- Binary search requires sorted data, so combining it with a sort is necessary if the data is unsorted.
- Insertion sort + binary search: slower than linear search for single-search cases because sorting takes most of the time (~2.1 s in your run).
- If multiple searches are performed on the same sorted list, the sorting overhead is amortised, making binary search advantageous.
- Binary search is extremely efficient on sorted data (O(log n)), but sorting an unsorted dataset beforehand can make it slower than linear search for a single search.
- Efficiency depends heavily on data reuse.